Data Handling and Infrastructure for AI

ML Foundamentals

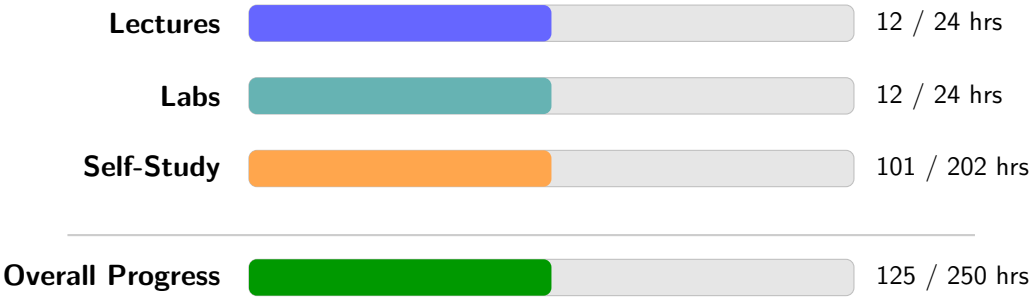
Tamás Grósz

SETU

2026



Workload


Lectures
12 / 24 hrs
Labs
12 / 24 hrs
Self-Study
101 / 202 hrs
Overall Progress
125 / 250 hrs

Learning Objectives

By the end of this lecture, you should be able to:

- Explain the basic idea behind supervised machine learning.
- Distinguish between common model families, e.g.:
  - Support Vector Machines (SVMs)
  - Deep Neural Networks (DNNs)
  - Transformers
- Explain the relationship between a learning objective and a loss function.
- Describe training using Stochastic Gradient Descent (SGD).
- Explain the meaning of a **step** and an **epoch**.
- Recognise overfitting and underfitting using learning curves.
- Explain the role of hyperparameters and hyperparameter optimisation.
- Explain why GPUs and TPUs accelerate modern ML training.
- Understand model cards and model versioning.

From Data to a Machine Learning Model

We already know how to:

- collect and store data,
- explore and clean data,
- construct dataflows and pipelines.

How do we turn this data into a model that can make useful predictions?

$$\boxed{\text{Data} \rightarrow \text{Model} \rightarrow \text{Prediction}}$$

But how do we choose the model parameters?

$$\boxed{\text{Training} = \text{Find parameters that perform well on the task}}$$

Machine Learning as Function Approximation

A machine learning model can be viewed as a parameterised function:

$$\hat{y} = f_\theta(x)$$

where:

- x = input features
- $\hat{y}$ = model prediction
- θ = learnable model parameters

Training attempts to find parameters θ that produce useful predictions.

$$\theta^* = \arg\min_\theta L(\theta)$$

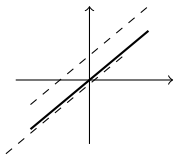
Key idea

The model architecture determines the form of f_θ. Training determines the values of θ.

Three Model Families

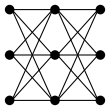
Today we will look at three important model families:

SVM



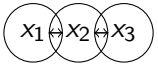
Decision boundary

DNN



Learned representations

Transformer



Attention and context

Different architectures make different assumptions and provide different trade-offs in computation, data requirements and capability.

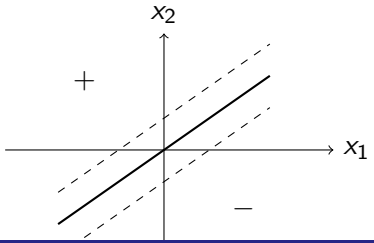
Support Vector Machines

An SVM attempts to find a decision boundary that separates classes.
For a linear classifier:

$$f(x) = w^T x + b$$

$$\hat{y} = \begin{cases} +1 & f(x) > 0 \\ -1 & f(x) < 0 \end{cases}$$

The SVM aims to find a boundary with a large **margin**.



Deep Neural Networks

A neural network composes many parameterised transformations.
A simple layer:

$$h = \sigma(Wx + b)$$

Multiple layers:

$$x \to h_1 \to h_2 \to \cdots \to h_L \to \hat{y}$$

where:

- W and b are learned parameters,
- σ is a nonlinear activation function,
- deeper networks can learn increasingly complex representations.

Why "deep"?

A DNN contains multiple trainable layers between the input and output.

Transformers

Transformers are neural network architectures built around **attention mechanisms**.
For an input: $x_1, x_2, \ldots, x_n$ the model can learn relationships between different positions.

A simplified attention operation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This allows the model to determine:

Which parts of the input are relevant to each other?

Transformers are widely used for:

- language,
- vision,
- multimodal models,
- sequence and time-series modelling.

Comparing the Model Families

	SVM	DNN	Transformer
Typical scale	Small–medium	Medium–large	Large–very large
Features	Often engineered	Learned	Learned
Nonlinearity	Kernel / boundary	Layers	Attention + layers
Training cost	Low–medium	Medium–high	Often very high
Typical use	Classification	Many domains	Sequences / multimodal

Important

There is no universally "best" model.
The appropriate architecture depends on the data, task, available compute and desired performance.

What Does "Learning" Mean?

Suppose we have training examples:

$$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

The model produces:

$$\hat{y}_i = f_\theta(x_i)$$

We need a way to measure how good the predictions are.

This is the role of the loss function.

Learning Objective and Loss Function

A loss function measures the error for a prediction.
For example, Mean Squared Error:

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

For classification, a common choice is cross-entropy:

$$L_{\text{CE}} = -\sum_{c} y_c \log(\hat{y}_c)$$

The overall learning objective is to minimise the loss on the training data:

$$\theta^* = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} L(f_\theta(x_i), y_i)$$

Loss Is Not the Same as the Evaluation Metric

The loss function is used during optimisation.
The evaluation metric tells us how useful the model is for the task.

Task	Possible loss	Possible metric
Regression	MSE	MAE / RMSE / R^2
Classification	Cross-entropy	Accuracy / F1
Ranking	Ranking loss	NDCG / MAP / Correlation
Object detection	Composite loss	mAP

Important

The metric we optimise is not necessarily the one we ultimately care about.

How Does the Model Learn?

We want to minimise:

$$L(\theta)$$

Gradient descent updates the parameters in the direction that reduces the loss:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

where:

- θ_t = current parameters
- η = learning rate
- $\nabla_\theta L$ = gradient of the loss

Compute gradient → update parameters → repeat

Stochastic Gradient Descent

Computing the gradient using the entire dataset can be expensive. Instead, SGD uses batches

$$B = \{(x_1, y_1), \ldots, (x_m, y_m)\}$$

The mini-batch gradient and update is approximately:

$$\nabla_\theta L \approx \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta L_i$$

$$\theta_{t+1} = \theta_t - \eta \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta L_i$$

Why mini-batches?

They provide a practical compromise between computational efficiency, memory usage and the statistical benefits of stochastic updates.

What Is a Training Step?

A **step** is one parameter update.
For one mini-batch:

$$\boxed{\text{mini-batch} \rightarrow \text{forward pass} \rightarrow \text{loss} \rightarrow \text{backpropagation} \rightarrow \text{parameter update}}$$

That is one training step.

For example:

$$\text{batch size} = 32$$

means that approximately 32 training examples contribute to each parameter update.

Remember

One step does **not** necessarily mean processing the entire dataset.

What Is an Epoch?

An **epoch** means that the training algorithm has processed the training dataset once.

$$N = 10\,000$$

training examples and:

$$B = 100$$

examples per batch, then:

$$\frac{N}{B} = \frac{10\,000}{100} = 100$$

steps are required for approximately one epoch.

$$\boxed{\text{steps per epoch} \approx \frac{\text{number of training examples}}{\text{batch size}}}$$

Training Loop

A simplified training loop:

1. Shuffle or sample training data.
2. Select a mini-batch.
3. Run the forward pass.
4. Calculate the loss.
5. Calculate gradients using backpropagation.
6. Update model parameters.
7. Repeat for all mini-batches.
8. Start another epoch.

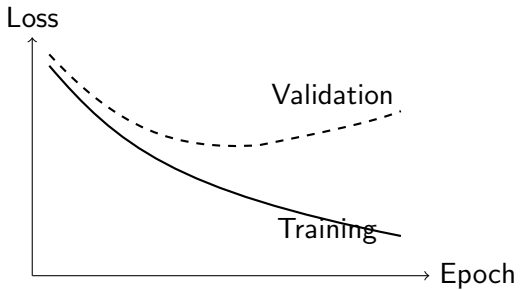
$$\boxed{\text{Data} \rightarrow \text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backward} \rightarrow \text{Update}}$$

Training and Validation Curves

During training we can record:

- training loss,
- validation loss,
- training metric,
- validation metric.

Plotting these values against training progress gives a **learning curve**.



Underfitting

A model is **underfitting** when it is unable to capture the patterns required by the task.
Typical symptoms:

- high training loss,
- high validation loss,
- both training and validation performance are poor,
- increasing training time does not solve the fundamental problem.

Possible causes:

- model is too simple,
- insufficiently informative features,
- excessive regularisation,
- insufficient training.

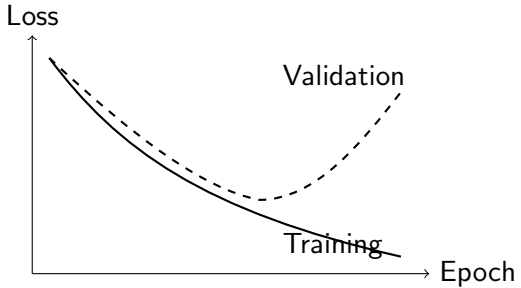
Intuition

The model has not learned enough of the underlying structure.

Overfitting

A model is **overfitting** when it learns the training data very well but generalises poorly.
Typical pattern:

- training loss continues to decrease,
- validation loss stops improving,
- validation loss may begin increasing.



How Can We Reduce Overfitting?

Possible approaches include:

- collect more training data,
- use data augmentation,
- reduce model complexity,
- use regularisation,
- use dropout in neural networks,
- use early stopping,
- improve the data split,
- tune the model and training configuration.

Infrastructure connection

Many of these techniques require repeatedly training and evaluating different versions of the model.

This is where experiment tracking and efficient compute become important.

The Learning Rate

The learning rate η controls the size of each parameter update:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

Learning rate too small

- Very small updates
- Training can be extremely slow
- May require many epochs

Learning rate too large

- Large parameter updates
- Loss may oscillate
- Training may become unstable

Learning rate = how large a step we take during optimisation

Important

The learning rate is usually one of the most important hyperparameters to get right when training a neural network.

Learning Rate Schedulers

We do not necessarily want the learning rate to remain constant throughout training. Instead, we can use a **learning rate scheduler**:

$$\eta_0 \to \eta_1 \to \eta_2 \to \cdots$$

For example, we might start with:

$$\eta = 10^{-3}$$

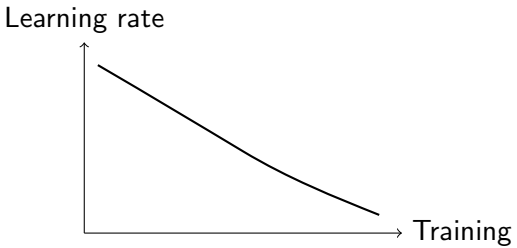
and gradually reduce it during training.

Common scheduling strategies include:

- **Step decay**: reduce the learning rate at predefined epochs.
- **Exponential decay**: gradually decrease the learning rate.
- **Cosine decay**: smoothly reduce the learning rate.
- **Reduce on plateau**: reduce it when validation performance stops improving.
- **Warm-up**: begin with a small learning rate and increase it initially.

Why Change the Learning Rate?

Early in training, we may want:**Larger updates** to move quickly towards a useful region of the parameter space.
Later in training, we may want: **Smaller updates** to make more precise adjustments.
A typical schedule might look like:

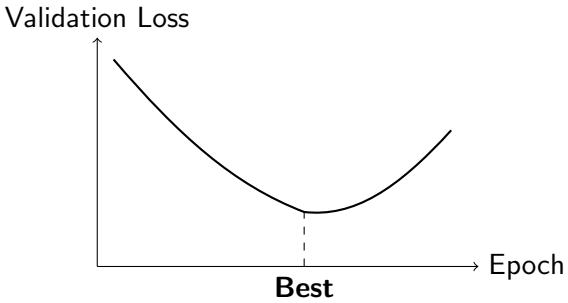


Connection to hyperparameter optimisation

The initial learning rate and the choice of scheduler are hyperparameters.

Early Stopping

Training for more epochs does not always improve the model.
Consider:



The validation loss reaches its best value and then starts to increase.
Early stopping terminates training when validation performance has stopped improving.

Stop training before the model starts to overfit

Early Stopping in Practice

A common approach is to define:

- a validation metric,
- a maximum number of epochs,
- a **patience** value.

$$\text{patience} = 5$$

Stop if validation performance does not improve for 5 epochs.

The best model checkpoint can be saved:

$$\boxed{\text{Training} \rightarrow \text{Validation} \rightarrow \text{Save best checkpoint} \rightarrow \text{Continue / stop}}$$

Infrastructure connection

Early stopping can save substantial compute when training large models or running many hyperparameter experiments.

Hyperparameters vs Parameters

It is important to distinguish:

Parameters

- Learned from the training data.
- Updated by optimisation.
- Examples:
  - neural network weights,
  - biases,
  - SVM weights.

Hyperparameters

- Chosen by the practitioner.
- Control the training process or model structure.
- Examples:
  - learning rate,
  - batch size,
  - number of layers,
  - regularisation strength.

Parameters are learned	Hyperparameters are configured

Hyperparameter Optimisation

Suppose we want to find:

$$\lambda^* = \arg\max_{\lambda} \mathrm{ValidationScore}(\lambda)$$

where λ represents a set of hyperparameters.
A basic workflow:

Choose configuration $\rightarrow$ Train $\rightarrow$ Evaluate $\rightarrow$ Record $\rightarrow$ Repeat

Common approaches:

- Grid search
- Random search
- Bayesian optimisation
- Population/evolutionary approaches

Example: Hyperparameters

Consider a DNN with:

- learning rate $\in \{10^{-2}, 10^{-3}, 10^{-4}\}$
- batch size $\in \{32, 64, 128\}$
- dropout $\in \{0, 0.2, 0.5\}$
- number of layers $\in \{2, 4, 6\}$

A grid search would require:

$$3 \times 3 \times 3 \times 3 = 81$$

training configurations.

If each training run takes 30 minutes:

$$81 \times 30 = 2430 \text{ minutes}$$

$$= 40.5 \text{ hours}$$

Why Do We Need GPUs?

Modern neural networks perform enormous numbers of numerical operations.
A large part of training consists of:

matrix multiplication

For example:

$$Y = XW$$

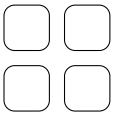
Many elements of Y can be calculated independently.

The computation is highly parallel.

CPUs are designed to handle a relatively small number of powerful, general-purpose cores.
GPUs contain many more specialised computational units designed to execute large numbers of similar operations in parallel.

CPU vs GPU — A Simplified View

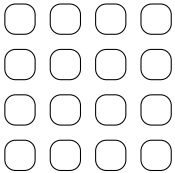
CPU



Fewer, powerful cores:

- general computation,
- control flow,
- diverse workloads.

GPU



Many parallel compute units:

- matrix operations,
- tensor operations,
- highly parallel workloads.

Important

A GPU does not automatically make every program faster. It is particularly useful when the workload exposes sufficient parallelism.

Why TPUs?

TPUs are specialised accelerators designed for machine learning workloads.
They are particularly suited to operations such as:

matrix multiplication and tensor computation

The broader principle is:

Specialised hardware $\rightarrow$ efficient execution of common ML operations

For modern AI systems, training may use:

- one GPU,
- multiple GPUs,
- GPU clusters,
- TPUs or other accelerators.

This connects directly to distributed and high-performance computing.

Why Does Hardware Matter to Training?

Suppose a training run requires: 10^{18} floating-point operations.
Reducing training time from:

$$10 \text{ days} \rightarrow 10 \text{ hours}$$

can change what is practically possible.
Faster hardware enables:

- larger models,
- larger datasets,
- more experiments,
- more hyperparameter trials,
- faster iteration.

The real benefit

Faster training is not only about saving time.
It changes the scale of experiments we can afford to perform.

But the GPU Is Not the Whole System

A GPU can only compute when it has data.

$$\text{Storage} \rightarrow \text{Data Loader} \rightarrow \text{CPU} \rightarrow \text{GPU} \rightarrow \text{Model}$$

If the GPU spends time waiting for data:

$$\boxed{\text{GPU utilisation} \downarrow}$$

Possible bottlenecks include:

- slow I/O,
- insufficient CPU preprocessing,
- network bandwidth,
- inefficient data loading,
- transferring data between devices.

Infrastructure connection

Efficient ML training requires the entire data-to-compute pipeline to work together.

A Typical ML Experiment

A training experiment might contain:

- Dataset version
- Data preprocessing version
- Model architecture
- Hyperparameters
- Training code version
- Hardware configuration
- Training metrics
- Validation metrics
- Model checkpoint

We want to be able to answer:

Why did this model perform better than the previous one?

Model Versioning

A model should not simply be:

`model.pt`

Model v1.0

Instead, think in terms of versions:

- Model architecture
- Training dataset version
- Code version
- Hyperparameters
- Training environment
- Evaluation results

Then:

$$\text{v1.0} \rightarrow \text{v1.1} \rightarrow \text{v2.0}$$

Each version should have enough information to understand how and why it was produced.

What Is a Model Card?

A **model card** is documentation describing a machine learning model.
It should communicate important information about:

- what the model is,
- what it was trained to do,
- what data was used,
- how it was evaluated,
- where it performs well,
- where it may fail,
- limitations and risks,
- intended and inappropriate uses.

Think of it as

A technical "README" for a trained model.

Example Model Card

Field	Example
Model	Image classifier v2.1
Task	Classify product images
Training data	Dataset v3
Architecture	ResNet-based DNN
Training	50 epochs, batch size 64
Metric	Accuracy / F1
Validation accuracy	94.2%
Intended use	Product catalogue classification
Known limitations	Poor performance on low-light images
Version	v2.1

The model card provides context that a model file alone cannot provide.

Why Versioning Matters

Imagine that six months from now: Model **accuracy suddenly decreases**
Questions:

- Did the data change?
- Did the preprocessing change?
- Did the model architecture change?
- Did the hyperparameters change?
- Did the training code change?
- Did the evaluation data change?

Without versioning:

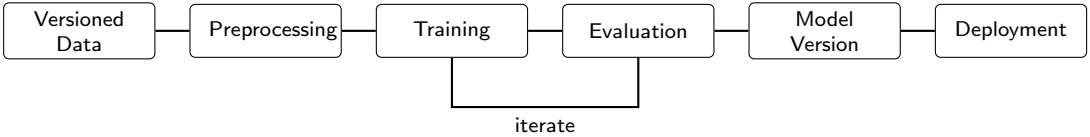
Debugging becomes guesswork

With versioning:

Changes can be traced and experiments reproduced

Connecting Everything Together

A practical ML workflow:



Each experiment should be traceable from:

$$\text{Data} \rightarrow \text{Code} \rightarrow \text{Configuration} \rightarrow \text{Training} \rightarrow \text{Model}$$

ML Training: The Big Picture

We can now connect the concepts:

1. Choose an appropriate model architecture.
2. Define the learning objective and loss.
3. Split the data appropriately.
4. Train using mini-batches and SGD.
5. Monitor training and validation performance.
6. Diagnose underfitting or overfitting.
7. Optimise hyperparameters.
8. Use appropriate compute infrastructure.
9. Record and version the resulting model.
10. Document the model using a model card.

Key Takeaways

- ML models are parameterised functions learned from data.
- SVMs, DNNs and Transformers represent different model families.
- A loss function gives the optimisation process a measurable objective.
- SGD updates model parameters using mini-batches.
- A **step** is one parameter update; an **epoch** is approximately one pass through the training data.
- Learning curves help diagnose underfitting and overfitting.
- Hyperparameters control how models and training behave.
- GPUs and TPUs accelerate highly parallel ML computations.
- Efficient training depends on the whole data-to-compute pipeline.
- Model versioning and model cards make ML systems reproducible and understandable.

Discussion / Questions

1. Why might a larger model perform worse on unseen data?
2. What happens if the learning rate is too large?
3. Why does increasing the batch size not simply make training proportionally faster?
4. Why can a powerful GPU still have low utilisation?
5. What information would you want to know before deploying a model trained by someone else?

Next: ML pipelines, MLOps and scalable training